# Lösungshinweise – Übungsserie Kapitel 1–4

Antworten auf die Fragen, typische Stolperstellen, Kontrollausgaben · Stand 03.11.2026

> **Hinweis**
> Erst lesen, wenn das Blatt bearbeitet ist.

Vollständiger Code liegt in `loesungen/blattN/…`. Alle Lösungen sind mit `-Wall -Wextra -pedantic` ohne Warnung übersetzt, alle Tests grün, valgrind ohne Meldung. Hier stehen die Antworten auf die Fragen und die Stellen, an denen man typischerweise hängen bleibt.

## Blatt 1

### Aufgabe 1 – zahlen / paare

- `kgv` liefert `long long`. Wichtig ist, erst zu teilen und dann zu multiplizieren (`a / g * b`), und zwar schon in `long long`. Sonst läuft `int` über.
- Negative Zahlen: am Anfang `std::abs`, das Ergebnis mit `std::llabs`.
- `ggt_rek(45, 10)` ruft `(10, 5)` und dann `(5, 0)` auf. Bei `b == 0` stehen **3** Aufrufe von `ggt_rek` auf dem Call-Stack.
- Bei `-zeit` sind die beiden Varianten etwa gleich schnell. gcc macht aus der Endrekursion mit `-O2` ohnehin eine Schleife. Mit `-O0` (Debug) ist die rekursive Variante meist etwas langsamer.

### Aufgabe 2 – werkzeug

- `element`: Rechne den Index mit `long`. Ist er negativ, addiere `len`. Prüfe danach `idx < 0 || idx >= len`. Vergleicht man `long` direkt mit `size_t`, wird −1 zu einer riesigen Zahl.
- `teile`: Prüfe den Divisor, **bevor** du schreibst. Nur so bleiben die Ausgabeparameter bei 0 unverändert.
- `zaehle(const std::string&, char c = ' ')`: Der Default-Wert steht nur im Header. `"Ene Mene Muh"` ist ein Literal. Daraus entsteht ein temporärer `std::string`, und an den bindet nur `const std::string&`. Mit `std::string&` übersetzt der Aufruf nicht (`cannot bind non-const lvalue reference … to an rvalue`).

### Aufgabe 3 – Orakel

| Zeile | Ergebnis | Begründung |
|---|---|---|
| A | 6 | 1 + 2 + 3, `::wert` ist das globale `wert` |
| B | 3 | globales `wert`. `using namespace std` stört nicht |
| C | 11 | `a::f(int)` |
| D | `21 11` | `f(1.5)`: `b::f(double)` passt exakt. `f(1)`: `a::f(int)` passt exakt |
| E | **Fehler** | `wert` ist mehrdeutig zwischen `a::wert`, `b::wert` und `::wert` |
| F | – | `int* const`: Der Wert, auf den gezeigt wird, darf geändert werden. `i` wird 11 |
| G | **Fehler** | `pc` selbst ist konstant |
| H | **Fehler** | Über `const int*` darf nicht geschrieben werden |
| I | – | `cp` darf umgesetzt werden, zeigt jetzt auf `k` |
| J | `int* const int* 8 const int* 8 12` | `&i` passt exakt zu `int*` und erhöht `i` auf 12. `&k` und `cp` (zeigt auf `k`) sind `const int*` |

| Zeile | Ergebnis | Begründung |
|---|---|---|
| K | `145 125 123` | Default-Werte von rechts |
| L | **Fehler** | `x` hat keinen Default-Wert |
| M | `0 9` | `groesser(12, 9)` liefert eine Referenz auf `i`, `i` wird 0 |
| N | **Fehler** | Das Literal 5 bindet nicht an `int&` |
| O | `OOP!` | `r` ist ein anderer Name für `s` |
| P | **Fehler** | Eine Referenz muss initialisiert werden |

Komplette Ausgabe ohne die fehlerhaften Zeilen: `6`, `3`, `11`, `21 11`, `int* const int* 8 const int* 8 12`, `145 125 123`, `0 9`, `OOP!` (jeweils eine Zeile).

**Aufgabe 4 – dreieck**

- Bytes für `n = 5` (64 Bit): 5 Zeiger zu 8 Byte = 40, dazu 1 + 2 + 3 + 4 + 5 = 15 `int` zu 4 Byte = 60, zusammen **100**. Allgemein: `n · sizeof(int*) + n(n+1)/2 · sizeof(int)`.
- Ohne `delete[] zeilen;` meldet valgrind `definitely lost: 40 bytes in 1 blocks`, also das Zeigerfeld. Ohne die Schleife meldet es `definitely lost: 60 bytes in 5 blocks`, also die Zeilen. Die Reihenfolge ist wichtig: zuerst die Zeilen freigeben, dann das Zeigerfeld. Umgekehrt würde man aus schon freigegebenem Speicher lesen.
- Mit `n > 20` passen die Zahlen nicht mehr in die Breite 6, deshalb die Grenze.

## Blatt 2

**Aufgabe 1 – Intervall**

- Konstruktoren: `Intervall(int von = 0)` delegiert an `Intervall(int von, int bis)`. Der zweite setzt `_von(min(von, bis)), _bis(max(von, bis))` in der Initialisierungsliste. Ein Default-Wert „bis = von“ geht nicht, weil ein Default-Wert keinen anderen Parameter verwenden darf. Deshalb braucht es Delegation.
- Der Konstruktor mit einem `int` ist bewusst **nicht** `explicit`, weil `c.ueberlappt(5)` funktionieren soll.
- `c.enthaelt(5)` ist **nicht** mehrdeutig. Gibt es beide Methoden, gewinnt `enthaelt(int)`, weil eine exakte Übereinstimmung jede benutzerdefinierte Umwandlung schlägt. Gibt es nur `enthaelt(const Intervall&)`, wird 5 implizit zu `[5, 5]`, und das Ergebnis ist dasselbe.
- `verschiebe` liefert `Intervall&` (`return *this;`), `vereinigung` ist `const` und liefert ein neues Objekt als Wert.
- Angrenzend heißt: `_bis + 1 == other._von`. Getrennt sind die Intervalle bei `_bis + 1 < other._von || other._bis + 1 < _von`.

**Aufgabe 2 – Belegungsplan / raum**

- Raumname und Öffnungszeit sind `const`-Attribute. Ein `const`-Attribut kann man nur in der Initialisierungsliste setzen, eine Zuweisung im Rumpf wäre ein Fehler.
- Ohne Standardkonstruktor von `Intervall` müsste **jedes** `Intervall`-Attribut in der Initialisierungsliste stehen. Sonst versucht der Compiler, es vor dem Rumpf mit dem Standardkonstruktor zu erzeugen, und meldet einen Fehler.
- `reservieren`: `_oeffnung.enthaelt(zeit)` und dann eine Schleife mit `ueberlappt`. Das erspart dir eigene Grenzvergleiche.
- `uhr(minute)`: `os << setfill('0') << setw(2) << minute / 60 << ":" << setw(2) << minute % 60;`. `setw` gilt nur für die nächste Ausgabe und muss deshalb zweimal stehen.

**Aufgabe 3 – Sensor**

- `_kennung` ist `const int` und wird in beiden Konstruktoren in der Initialisierungsliste mit `neue_kennung()` gesetzt. Die Hilfsmethode erhöht `_aktive`, aktualisiert `_hoechststand` und

liefert `_naechste++`.

- Der Zuweisungsoperator übernimmt nur `_messwert`. Den automatisch erzeugten gibt es wegen des `const`-Attributs gar nicht, er wäre `deleted`.
- Ausgabe der Verfolgung (gcc/libstdc++):

```
Start: aktive 0, hoechststand 0
nach push_back: aktive 1, hoechststand 2
nach push_back: aktive 2, hoechststand 4
nach push_back: aktive 3, hoechststand 6
Kennungen im vector: 8 9 7
nach Block 1: aktive 0, hoechststand 6
Block 2 mit reserve: aktive 3, hoechststand 6
Ende: aktive 0, hoechststand 6
```

Erklärung: Jedes `push_back(Sensor(…))` erzeugt einen temporären Sensor und eine Kopie im Vektor. Weil `Sensor` einen eigenen Kopierkonstruktor hat, wird kopiert, nicht verschoben. 1. Aufruf: temporär (1), Kopie (2). Gleichzeitig existieren 2, also Höchststand 2. 2. Aufruf: temporär (3). Der Vektor ist voll und wächst auf Kapazität 2. Zuerst wird das neue Element kopiert (4), dann das alte umkopiert (5). Danach sterben 2 und 3. Kurzzeitig existieren 4 Sensoren. 3. Aufruf: temporär (6), neues Feld der Kapazität 4. Das neue Element wird kopiert (7), die alten werden umkopiert (8, 9). Kurzzeitig existieren 6 Sensoren. Im Vektor stehen deshalb 8, 9, 7.

Mit `reserve(3)` wird nie umkopiert. Es existieren höchstens 3 + 1 temporärer Sensor gleichzeitig, also bleibt der Höchststand 6.

**Aufgabe 4 – kompakt**

```
struct Farbe {
    int r, g, b;
    Farbe(int w = 0) : Farbe(w, w, w) {}
    Farbe(int r, int g, int b = 0) : r(r), g(g), b(b) {}
    Farbe(char c) : Farbe(c == 'r' ? 255 : 0, c == 'g' ? 255 : 0, c == 'b' ? 255 : 0) {}
};
```

Ohne `Farbe(char)` würde `'r'` zum `int` 114 befördert, und die Ausgabe wäre `114,114,114`.

```
class Zaehlwerk {
public:
    explicit Zaehlwerk(int start = 0) : _stand(start) {}
    Zaehlwerk& tick() { ++_stand; ++_ticks; return *this; }
    int stand() const { return _stand; }
    static int ticks() { return _ticks; }
private:
    int _stand;
    static int _ticks;
};
int Zaehlwerk::_ticks = 0;
```

`c.tick()` scheitert mit `discards qualifiers`, `Zaehlwerk d = 7;` mit `conversion from 'int' to non-scalar type 'Zaehlwerk' requested`.

## Blatt 3

**Aufgabe 1 – Bitmenge**

- Bit setzen: `_bytes[i / 8] |= (1u << (i % 8))`, löschen: `&= ~(1u << (i % 8))`, lesen: `(_bytes[i / 8] >> (i % 8)) & 1u`.
- `new unsigned char[k]{}` legt alle Bytes mit 0 an.
- Bitmuster nach `setze(1); setze(8);` (Bit 7 links): Byte 0 = `00000010`, Byte 1 = `00000001`.

- Copy & Swap: `operator=(Bitmenge other)` erhält bereits eine Kopie. Danach `swap(*this, other)`, und der alte Speicher stirbt mit `other`. Bei Selbstzuweisung wird erst kopiert und dann getauscht, also ist nichts zu prüfen.
- Kontrolle: Mit flacher Kopie im Kopierkonstruktor scheitern die Tests `Kopie` und `Zuweisung`.

**Aufgabe 2 – Zeilenpuffer / rueckwaerts**

- `new std::string[n]` ruft für jedes Element den Standardkonstruktor von `std::string` auf, `delete[]` jeden Destruktor. `delete` statt `delete[]` wäre hier undefiniertes Verhalten und führt meist zum Absturz.
- `entferne`: Schleife `k = i … size − 2` mit `_zeilen[k] = _zeilen[k + 1]`, danach `--_size`.
- `for (size_t k = puffer.size() - 1; k >= 0; --k)` ist eine **Endlosschleife**: Ein `size_t` ist nie negativ, nach 0 kommt die größte Zahl. Bei leerem Puffer ist schon der Startwert riesig, und `zeile(k)` wirft. Richtig ist `for (size_t k = size; k > 0; --k)` mit dem Zugriff auf `k − 1`.

**Aufgabe 3 – CsvLeser**

- Attribute: `std::ifstream _datei; const char _trenner; size_t _zeile, _gelesen;`. Den Stream öffnest du in der Initialisierungsliste mit `_datei(dateiname)`, danach prüfst du `if (!_datei)`.
- `= delete` für Kopierkonstruktor und Zuweisung. `ifstream` ist ohnehin nicht kopierbar, aber so ist die Absicht ausdrücklich und die Fehlermeldung klar.
- Einen eigenen Destruktor brauchst du **nicht**. Nach dem (leeren) Destruktor von `CsvLeser` werden die Attribute in umgekehrter Reihenfolge zerstört (Skript 4.1). Der Destruktor von `ifstream` schließt die Datei.
- Leeres letztes Feld: Nach der Schleife über die Zeichen wird das letzte Feld **immer** angehängt, auch wenn es leer ist.

**Aufgabe 4 – Puffer**

1. Nach Zeile 6: `a._p` und `c._p` zeigen auf dasselbe Feld `{9, 0, 0}` (flache Kopie in Zeile 4). Durch die Zuweisung in Zeile 6 zeigt auch `b._p` dorthin, und `b._n` ist 3. Das ursprüngliche Feld von `b` (5 `int`) zeigt niemand mehr an.
2. Zeile 7 gibt `9 3` aus. `c.at(0) = 9` hat das gemeinsame Feld geändert.
3. 5 · 4 = **20 Byte** gehen in Zeile 6 verloren. valgrind meldet `definitely lost: 20 bytes in 1 blocks`.
4. Die Destruktoren laufen in umgekehrter Reihenfolge: `c`, `b`, `a`. Die Ausgabe ist `D3 D3` und dann der Abbruch `free(): double free detected`. Das Programm stürzt also im Destruktor von `b` ab, dem zweiten `delete[]` auf denselben Zeiger.
5. Mit Kopierkonstruktor (tiefe Kopie, Ausgabe `C`) und Copy & Swap lautet die Ausgabe:

```
K3 K5 C3 C3 D5 7 3
D3 D3 D3
```

Zeile 4 erzeugt `C3`. Zeile 6 erzeugt `C3` für die Kopie im Wertparameter, danach `D5`, wenn der Parameter mit dem alten Speicher von `b` stirbt. `a.at(0)` bleibt 7.